Software Project Estimation

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

innovative • entrepreneurial • global

Outline

- Understanding Software Effort Estimation Basics
- Factors Affecting Software Development Costs
- Proven Techniques for Estimating Project Budgets
- Top-Down Estimation
- Bottom-Up Estimation

Learning Outcomes

By the end of this chapter, you will be able to:

- Understand the fundamentals of software effort and cost estimation
- Explain key estimation concepts such as effort, cost, size, and complexity
- Compare traditional estimation models with agile-based estimation approaches
- Apply estimation techniques such as T-Shirt,Velocity,Function Point and others.
- Recognize factors that influence estimation accuracy
- Appreciate the importance of estimation in planning, budgeting, and monitoring software projects

Understanding Software Effort Estimation Basics



What is Software Effort Estimation?

Software Effort Estimation refers to predicting the amount of work, time, and resources required to develop a software product.

- Helps determine project feasibility
- Supports budgeting and scheduling
- Essential for planning releases in both traditional and agile projects
- Mentioned as a critical early-stage activity in research (Rivera Ibarra et al., 2024)



Purpose of Software Effort Estimation



Estimation provides a foundation for making informed decisions about project planning and delivery. It reduces uncertainty and helps teams prepare for the work ahead.

- Plan project timelines
- Allocate team members and resources
- Estimate cost and effort
- Reduce project risks and uncertainties
- Support decision-making for contracts and budgeting

(Reference: Mike Cohn, Agile Estimating and Planning)

Importance of Software Estimation



Accurate estimation improves planning and ensures projects deliver value within realistic constraints. It helps both teams and stakeholders set expectations.

- Prevents overruns in cost and schedule
- Enables better resource planning
- Helps manage stakeholder expectations
- Supports delivery of high-value features

Challenges in Estimation

Software development involves uncertainty, changing requirements, and human judgement — all of which make estimation difficult.



- Requirements may be unclear early
- Technical complexity is hard to predict
- Human bias affects estimates
- Teams may lack historical data
- Agile projects constantly evolve

Effort vs Duration vs Cost

Although related, effort, duration, and cost measure different aspects of a project and should not be confused.

- Effort: Human labor required
- Duration: Calendar time including delays
- Cost: Monetary value based on effort and resources
- Effort does not always equal duration



Factors Affecting Software Development Costs



What Effects Software Development Costs?

Software development costs are influenced by many internal and external factors. These factors impact effort, timeline, resources, and estimation accuracy across projects.

- Product characteristics
- Project environment
- Technical complexity
- Human and team factors
- Organizational maturity
- Lessons from empirical studies



Product Factors



Product-related attributes significantly influence how much effort a project requires. Larger or more complex products naturally demand more time and resources.

- System size (SLOC, FP, Story Points)
- Functional complexity (features, interfaces)
- Required performance, reliability, and security
- Number of modules or integrations

Project Environment Factors

The environment surrounding the project also affects cost. These conditions shape how smoothly development can proceed.

- Tools, frameworks, and technologies used
- Agile vs traditional environment differenc
- Availability of domain experts
- Requirements clarity or volatility
- Contract type (flexible vs fixed) impacts estimation variation



Technical Complexity Modeling



Technical complexity is a major cost driver because complex software demands more effort in design, coding, testing, and integration.

- Complex algorithms or data structures
- Multiple system interfaces
- Security and compliance needs
- Integration with legacy systems
- Hidden complexity increases effort (Scrum SLR finding)

Human & Team Factors

People-related factors play a major role. Skills, communication, and team structure have direct effects on cost and productivity.

- Team experience influences accuracy and productivity (ranked highest factor)
- Skill level with programming tools and languages
- Communication overhead (Brooks's Law effect)
- Coordination challenges in larger teams
- Team cohesion impacts COCOMO scale factors (TEAM)



Time & Scheduling Factors

How time is allocated affects costs—both in agile and traditional environments.

- Shorter, iterative cycles reduce large uncertainty
- Delays in one stage influence downstream tasks
- Verification & validation effort distribution differs (Agile vs Traditional)
- Changing deadlines add cost pressures



Organizational Maturity



Organizations with well-established processes experience lower cost variation.

- Use of standards (CMMI, structured processes)
- Experience handling Agile in acquisition environments
- Better risk identification and management
- Clear communication between teams and stakeholders

Relevant insight from Agile acquisition study:
Organizations lacking maturity struggle with estimation consistency.

Summary: Combined Cost Influencing Factors

Software costs are shaped by many dimensions that interact across the product, team, and organization.

- Larger and more complex products cost more
- Experienced teams estimate and deliver more accurately
- Organizational maturity enhances predictability
- Technical complexity drives testing and integration cost
- Agile processes reduce uncertainty but require team alignment
- Research shows experience, complexity, time, and size as core drivers

All factors must be considered for realistic software cost estimation.

Proven Techniques for Estimating Project Budgets



What Are Budget Estimation Techniques?

Estimation techniques help teams predict cost and effort more accurately before development begins. They provide structure so teams don't rely on guesswork. Different techniques suit different levels of detail and project types.

- Improve prediction of effort and cost
- Reduce uncertainty during planning
- Support informed decision-making



Why These Techniques Matter



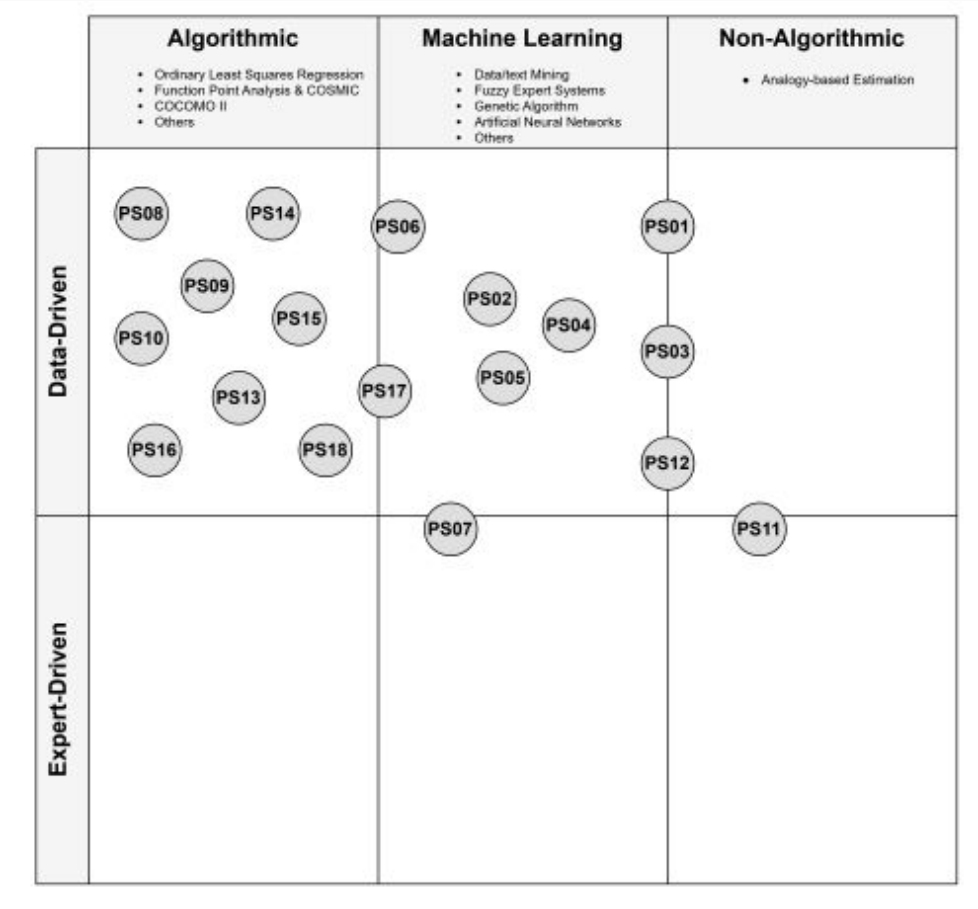
Using proven estimation methods helps teams avoid underestimation and project overruns. Teams gain clearer understanding of workload and risks. These techniques also improve planning confidence for stakeholders.

- Avoid unrealistic commitments
- Identify risk early
- Improve communication with stakeholders

Categories of Estimation Techniques

Agile estimation methods fall into several major categories. Each category offers a different way to measure effort or size. Understanding them helps teams choose the right method for budgeting and planning.

- Story-based / relative estimation
- Time-based estimation
- Probabilistic / flow forecasting
- Expert-judgment methods
- Hybrid / model-assisted approaches



Top-Down Estimation



What Is Top-Down Estimation?

Top-down estimation starts with a big-picture view of the project and gives a quick overall effort estimate. It is used early when details are not fully known.

- Estimate from overall system
- Fast and simple
- Used early in planning
- Uses T-shirt sizing & high-level Story Points



Types of Top-Down Estimation



Top-down estimation uses visible features of the product to estimate effort based on size groups and similarity.

- T-Shirt Estimating (group items into S, M, L, XL)
- Function Point Estimating
- Velocity-Based Estimation

1. T-Shirt Estimation

What Is T-Shirt Sizing?

T-shirt sizing assigns categories like XS, S, M, L, and XL to represent effort levels. It is a coarse estimation method used when details are not clear. Teams often use it for epics or early-stage planning.

- Uses XS–XL categories
- Very simple and fast
- Ideal for high-level estimation



1. T-Shirt Estimation



Why Teams Use T-Shirt Sizing

Teams use T-shirt sizing because it avoids over-analysis and quickly shows which items are large or small. It helps stakeholders understand scope without technical detail. Later, teams convert sizes into Story Points.

- Quick scope comparison
- Good for discussions with stakeholders
- Helps identify large epics early

2. Velocity-Based Estimation

What Is Velocity-Based Release Planning?

Velocity measures how many Story Points a team completes each sprint. Release planning uses this velocity to forecast how long it will take to finish the product backlog. It becomes more accurate after several sprints.

- Based on completed Story Points
- Used to predict release timelines
- Improves as team consistency grows



2. Velocity-Based Estimation



Why Teams Use Velocity

Teams use velocity because it provides data-driven forecasting instead of guesswork. It allows predictable budgeting and helps identify when a scope change affects delivery. Velocity also supports continuous improvement.

- Helps forecast future work
- Tracks changes in team performance
- Supports realistic planning conversations

3. Function Point Estimation

What Is a Function Point?

A Function Point measures the amount of functionality a system provides to the user. It counts things like inputs, outputs, and data files to show how big the system is. It does not depend on programming language.

- Measures system size
- Based on user-visible functions
- Independent of technology



3. Function Point Estimation



Why Teams Use Function Points

Teams use Function Points to estimate work early before coding begins. FP helps them understand system size and plan cost or effort. It gives a clear and consistent way to compare projects.

- Measures system size
- Based on user-visible functions
- Independent of technology

3. Function Point Estimation



Components of Function Points

FP is calculated by identifying different types of system functions. Each function type contributes a certain number of points depending on complexity. These components reflect how data and processes flow in the system.

- External Inputs (EI)
- External Outputs (EO)
- External Inquiries (EQ)
- Internal Logical Files (ILF)
- External Interface Files (EIF)

FUNCTION POINT - CASE STUDY 1

Employee Leave Request System

This case study represents a small internal HR system that allows employees to apply for leave, check their balance, and view approval status. The system has basic input forms, simple outputs, and a single database file to store leave records. It is ideal for demonstrating how Function Points are counted in a small-sized project.

- Step A — Identify and count components (example)

  1. External Inputs (EI): 3 (leave request, cancel, modify) → assume Average
  2. External Outputs (EO): 2 (leave summary, notification) → Average
  3. External Inquiries (EQ): 1 (leave balance query) → Average
  4. Internal Logical Files (ILF): 1 (leave records DB) → Average
  5. External Interface Files (EIF): 0

FUNCTION POINT - CASE STUDY 1

- Step B — Apply complexity weights & compute UFP
  Using the standard weights above:
  EI: 3 × 4 = 12
  EO: 2 × 5 = 10
  EQ: 1 × 4 = 4
  ILF: 1 × 10 = 10
  EIF: 0 × … = 0
  UFP = 12 + 10 + 4 + 10 = 36

- Step C — Value Adjustment Factor (VAF)
  Assume the system has ΣGSC = 20 (moderate environment).
  VAF factor = 0.65 + 0.01 × 20 = 0.65 + 0.20 = 0.85
  FP = UFP × VAF = 36 × 0.85 = 30.6 → round to 31 FP

FUNCTION POINT - CASE STUDY 1

- Step D — Convert FP to effort (example productivity rates)

  Two example productivity rates (these depend on organization; adjust as needed):
  Productivity A: 10 FP / person-month → Effort = 31 / 10 = 3.1 person-months
  Productivity B (slower): 7 FP / person-month → Effort = 31 / 7 ≈ 4.4 person-months

  Answer
  Function Points ≈ 31 FP.
  Estimated effort ≈ 3.1 PM (at 10 FP/PM) or 4.4 PM (at 7 FP/PM).
  If you want a calendar duration: with a 2-person team → ~1.6–2.2 months (approx).

FUNCTION POINT - CASE STUDY 2

Online Ordering System

This case study describes a medium-sized e-commerce application used for browsing products, placing orders, processing payments, and tracking shipping. The system includes several user interactions, multiple files, external integrations, and moderate functional complexity. It provides a realistic example of how Function Points scale with system size.

FUNCTION POINT - CASE STUDY 2

- Step A — Identify and count components (example)

1. External Inputs (EI): 5 (login, add to cart, checkout, update profile, payment input)
   We assume a mix; treat most as Average.
2. External Outputs (EO): 6 (order confirmation, invoice, order history, email notif, shipping status, reports) → Average
3. External Inquiries (EQ): 4 (order lookup, product search, price check, stock check) → Average
4. Internal Logical Files (ILF): 3 (product catalog, orders DB, customers DB) → Average
5. External Interface Files (EIF): 2 (payment gateway, shipping provider) → Average

FUNCTION POINT - CASE STUDY 2

- Step B — Apply complexity weights & compute UFP

  Using typical Avg weights:
  EI: 5 × 4 = 20
  EO: 6 × 5 = 30
  EQ: 4 × 4 = 16
  ILF: 3 × 10 = 30
  EIF: 2 × 7 = 14
  UFP = 20 + 30 + 16 + 30 + 14 = 110
- Step C — Value Adjustment Factor (VAF)

  Assume ΣGSC = 35 (richer functionality + quality/performance needs).
  VAF = 0.65 + 0.01 × 35 = 0.65 + 0.35 = 1.00
  FP = UFP × VAF = 110 × 1.00 = 110 FP

FUNCTION POINT - CASE STUDY 2

- Step D — Convert FP to effort (example productivity)

  Two example productivity rates:
  1. Productivity A: 10 FP/PM → Effort = 110 / 10 = 11.0 person-months
  2. Productivity B: 7 FP/PM → Effort = 110 / 7 ≈ 15.7 person-months

  If you plan a 4-person team:
  1. At 10 FP/PM → duration ≈ 11 / 4 = 2.8 months
  2. At 7 FP/PM → duration ≈ 15.7 / 4 = 3.9 months

  Answer

  Function Points = 110 FP.
  Estimated effort ≈ 11 PM (10 FP/PM) or 15.7 PM (7 FP/PM).
  Calendar duration depends on team size (examples above).

Bottom-Up Estimation



What Is Bottom-Up Estimation?

Bottom-up estimation starts by breaking work into small tasks and estimating each one. It is used when the team has clear details of the work.

- Estimate from small tasks
- More accurate
- Used later when details are known
- Uses task breakdown & detailed sizing



Types of Bottom-Up Estimation



Bottom-up estimation breaks work into smaller pieces and estimates each one separately.

- Work Breakdown Estimating (estimate each task)
- Product Breakdown Estimating

1. Product Breakdown Estimating

What is Product Breakdown Estimating

Product Breakdown Estimating divides the system into smaller components or sub-products. Each component is then estimated separately. This approach is helpful when dealing with large or complex systems.

- Breaks work into smaller tasks
- Estimates each task individually
- Adds tasks to get total effort



1. Product Breakdown Estimating



Why Teams Use Product Breakdown Estimating

Teams use Product Breakdown Estimating to simplify large features into manageable pieces. It helps ensure no major part of the system is forgotten. This method improves clarity and planning accuracy for big projects.

- Simplifies complex systems
- Ensures complete coverage of components
- Improves planning for large features

2. Work Breakdown Estimating

What Is Work Breakdown Estimating?

Work Breakdown Estimating breaks a feature into small, clear tasks and estimates each task separately. It helps teams understand the detailed work involved before development begins. The total effort is calculated by adding all task estimates together.



- Breaks work into smaller tasks
- Estimates each task individually
- Adds tasks to get total effort

2. Work Breakdown Estimating



Why Teams Use Work Breakdown Estimating

Teams use Work Breakdown Estimating because it gives more accurate estimates and reduces missed work. It also helps teams see what needs to be done in detail. This method is especially useful during sprint planning.

- Produces more accurate estimation
- Helps identify all required tasks
- Useful for detailed sprint planning

WORK BREAKDOWN - CASE STUDY 1

Create a Login Screen

This case study shows how a small feature can be broken into simple tasks. Each task is estimated separately, and the total effort is the sum of all tasks.

- Task Breakdown
  1. Review UI standards → 1 hour
  2. Design login layout → 2 hours
  3. Build login UI → 3 hours
  4. Connect to database → 2 hours
  5. Test login functionality → 2 hours

Total Effort : *1+2+4+2+3=**12 hours.***

WORK BREAKDOWN - CASE STUDY 2

Create Course Search & Filter Feature

This feature allows students to search for courses using keywords and filter them by category (e.g., department, credit hours, level). It is more complex than a basic form but not as heavy as a full module—perfect for a medium case study.

- Task Breakdown
  - A. UI & Front-End Tasks
    i. Design search bar + filter dropdowns → 2 hours
    ii. Build search input and filter components → 3 hours
    iii. Display search results list → 3 hours
    Subtotal: 8 hours

WORK BREAKDOWN - CASE STUDY 2

- B. Backend & Logic Tasks
  i. Create API to fetch course list → 3 hours
  ii. Add keyword search logic → 3 hours
  iii. Add filter logic (department, credits, level) → 3 hours
  Subtotal: 9 hours

- C. Database Tasks
  i. Add indexes to improve search speed → 2 hours
  ii. Create SQL query for search + filters → 2 hours
  Subtotal: 4 hours

WORK BREAKDOWN - CASE STUDY 2

- D. Testing Tasks
  i. Test search function with sample data → 2 hours
  ii. Test filters (department, credit hours, level) → 2 hours
  iii. Fix bugs found during testing → 2 hours

  Subtotal: 6 hours

  TOTAL EFFORT= *8+9+4+6=**27 hours***

INTERPRETATION

- 1 developer: ~3 working days
- 2 developers: ~1.5 days each
- Medium complexity: more steps, but not overwhelming



THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

www.utm.my